

LECTURE 8

Introduction to Threads and Concurrency

CSC323 · Operating Systems

COMSATS University Islamabad, Lahore Campus

In This Lecture

- What a thread is, and exactly what it shares with its sibling threads vs. what it keeps private
- Why threads exist: responsiveness, resource sharing, economy, and scalability
- Processes vs. threads, compared directly
- Concurrency vs. parallelism — a frequently confused but genuinely different distinction
- Amdahl's Law: the formula bounding how much speedup more cores can actually deliver

One Process, Many Things Happening at Once

A modern web browser renders a page, plays a video, and responds to your typing all at what feels like the same time, without spawning a brand-new, fully isolated process for each job. It does this with threads — a second, finer-grained unit of execution living inside a single process.

What Is a Thread?

A thread is a lightweight unit of execution within a process. Like a process, it has its own program counter, its own CPU registers, and its own stack — everything it needs to be independently scheduled and run.

- Unlike a process, a thread does not get its own private copy of everything else
- All threads of the same process share that process's code (text section), its global data, its heap, and its open files

One Process, Three Threads — Shared vs. Private

Shared across all threads

Code (text section), global data, heap, open files

Private to each thread

Program counter, CPU registers, its own stack

A house, and the people living in it. Each thread has its own bedroom (its own stack, its own registers — private, personal space) but all threads share the same kitchen, the same living room, and the same front door (the process's code, data, and open files).

Why Threads?

If a process can already do work, why introduce a second, smaller unit of execution inside it? Four reasons that show up constantly in real software:

Responsiveness

A program keeps responding to the user even while part of it is busy — keystrokes still register while a spell-check runs in the background

Resource sharing

Threads of the same process automatically share memory and files, without the deliberate IPC that separate processes need

Economy

Creating a thread, and switching between two threads of the same process, is cheaper than a whole new process — no private address space to set up

Scalability

On a multicore machine, the threads of a single process can genuinely run at the same time on different cores

Processes vs. Threads

	Process	Thread
Address space	Its own, isolated from every other process	Shared with every other thread in the same process
Creation cost	Heavier — new memory tables, new PCB, new file structures	Lighter — reuses the parent process's existing structures
Context-switch cost	Heavier — may involve switching memory mappings	Lighter — registers and stack pointer change, address space does not
Communication with siblings	Requires IPC (shared memory or message passing)	Direct — simply read or write the same shared memory
Failure isolation	A crashing process does not corrupt another process's memory	A misbehaving thread can corrupt data shared with sibling threads

Shared Memory Between Threads Is a Double-Edged Sword

The same property that makes threads cheap is what makes them dangerous. Direct, unmediated access to shared data is convenient, but two threads reading and writing the same shared variable at the same time face precisely the synchronization problem flagged for shared-memory IPC — except now it's the default, not an opt-in choice. A later unit of this course is devoted entirely to keeping concurrent access to shared data correct.

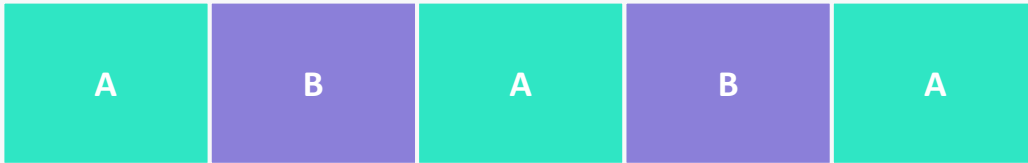
Concurrency vs. Parallelism

- Concurrency means multiple tasks are making progress over some interval of time, without necessarily running at the exact same instant — a single core can interleave slices of thread A, then B, then back to A
- Parallelism means multiple tasks are running literally at the same instant, which requires multiple physical execution units — multiple CPU cores, each actually executing a different thread's instructions simultaneously

Interleaved vs. Simultaneous

Concurrency — one core

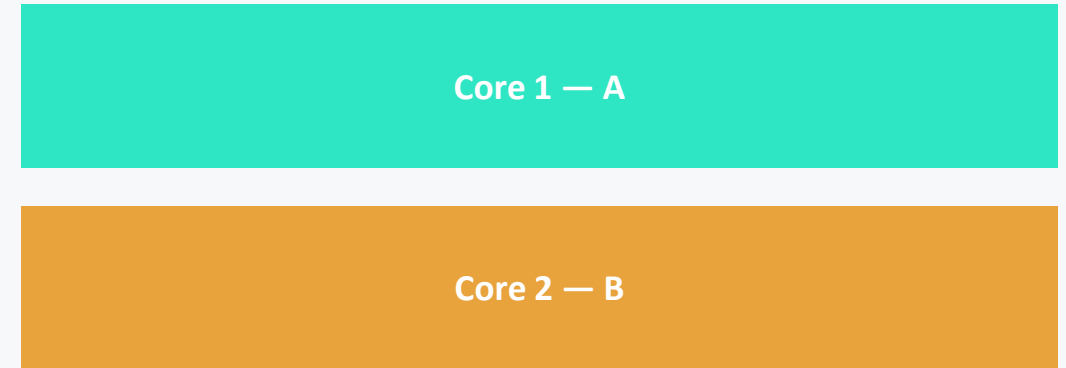
Thread A and Thread B interleaved — never truly simultaneous



CPU → time

Parallelism — two cores

Thread A on Core 1 and Thread B on Core 2, genuinely simultaneous



Every parallel system is concurrent, but not every concurrent system is parallel. A single-core machine running a multithreaded program is concurrent but never truly parallel. A multicore machine can be genuinely parallel, but only if the scheduler actually places different threads on different cores at the same moment.

Amdahl's Law

Once a program has multiple threads that can genuinely run in parallel, the obvious question is: how much faster does adding more cores actually make it? Amdahl's Law answers that — and the answer is less generous than intuition suggests.

$$\text{speedup} \leq 1 / (S + (1 - S) / N)$$

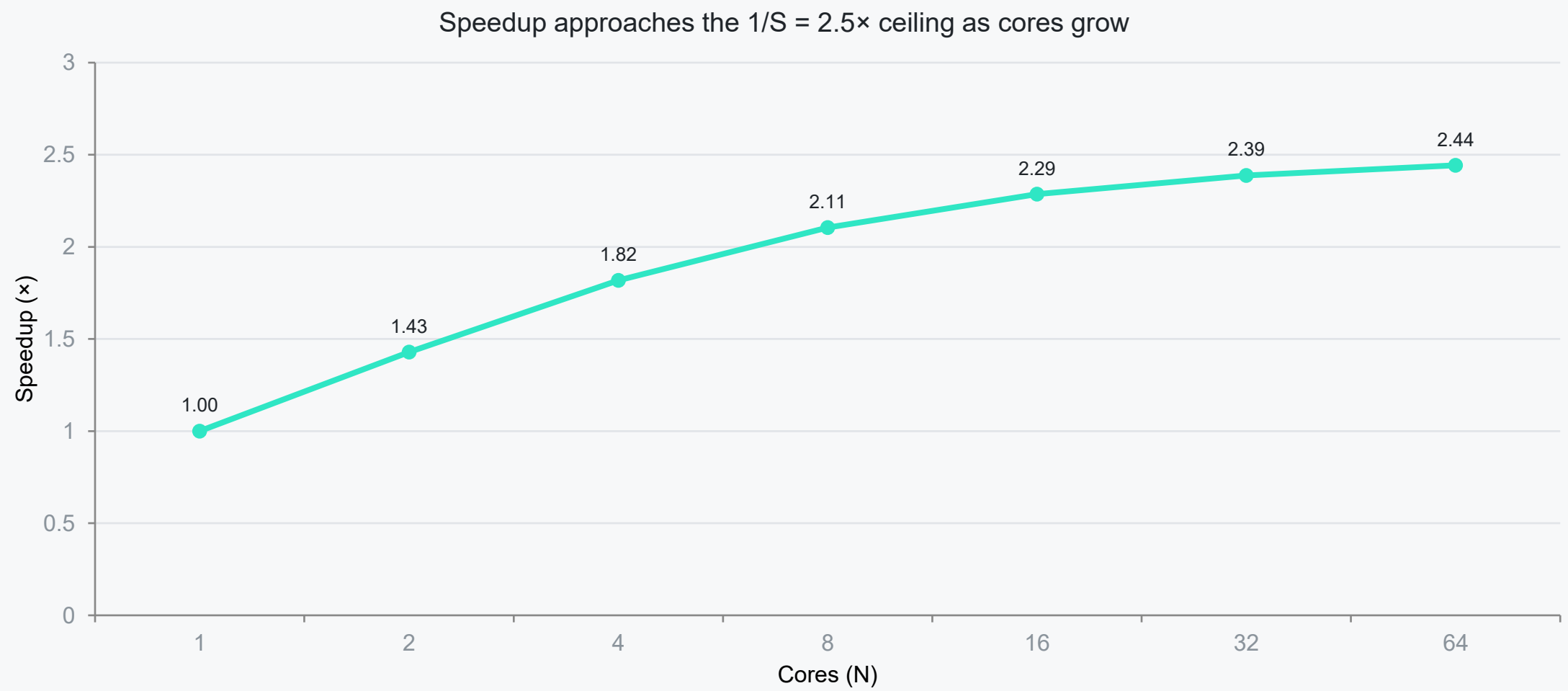
- S is the fraction of the program that must run serially
- (1 - S) is the fraction that can be parallelized
- N is the number of processor cores available

Worked Example — a Program That Is 40% Serial (S = 0.4)

Cores (N)	Calculation	Speedup
2	$1 / (0.4 + 0.6/2) = 1 / 0.70$	1.43×
4	$1 / (0.4 + 0.6/4) = 1 / 0.55$	1.82×
8	$1 / (0.4 + 0.6/8) = 1 / 0.475$	2.11×
∞	$1 / (0.4 + 0.6/\infty) = 1 / 0.4$	2.50×

Notice how little is gained between N = 4 and N = 8. Even with an infinite number of cores, the speedup never exceeds 2.5×, because the parallel term $(1 - S)/N$ shrinks toward zero, leaving the speedup bound entirely by the serial fraction: $1/S$.

Speedup vs. Cores — Amdahl's Law ($S = 0.4$)



The Headline Example: 25% Serial Caps You at 4×

Run the same formula with $S = 0.25$: the maximum possible speedup is $1/0.25 = 4\times$ as $N \rightarrow \infty$ — no matter whether you add 8 cores, 64 cores, or 1,000 cores. The serial portion of a program, however small, sets a hard ceiling on parallel speedup that more hardware alone can never break through. The only way past that ceiling is to reduce S itself.

Why “Just Add More Cores” Eventually Stops Working

It's tempting to assume doubling the cores roughly doubles performance. Amdahl's Law shows that assumption breaks down fast once any serial portion exists at all — the worked example gained only another 0.29× speedup going from 4 cores all the way to infinity. Identifying and shrinking a program's serial fraction is usually a far more productive investment than buying more cores for a program that already has a significant one.

Key Takeaways

- A thread is a lightweight unit of execution inside a process, with its own program counter, registers, and stack, but sharing code, data, heap, and open files with every sibling thread
- Threads exist for responsiveness, resource sharing, economy, and scalability across multiple cores
- Processes give strong isolation at a heavier cost; threads are cheap and share memory directly, which removes the OS-enforced boundary that kept processes from corrupting each other
- Concurrency is multiple tasks making progress, possibly interleaved on one core; parallelism is multiple tasks executing at the literal same instant on multiple cores
- Amdahl's Law, $\text{speedup} \leq 1 / (S + (1-S)/N)$, bounds the maximum possible speedup by the serial fraction S — a program that's 40% serial can never exceed a 2.5× speedup